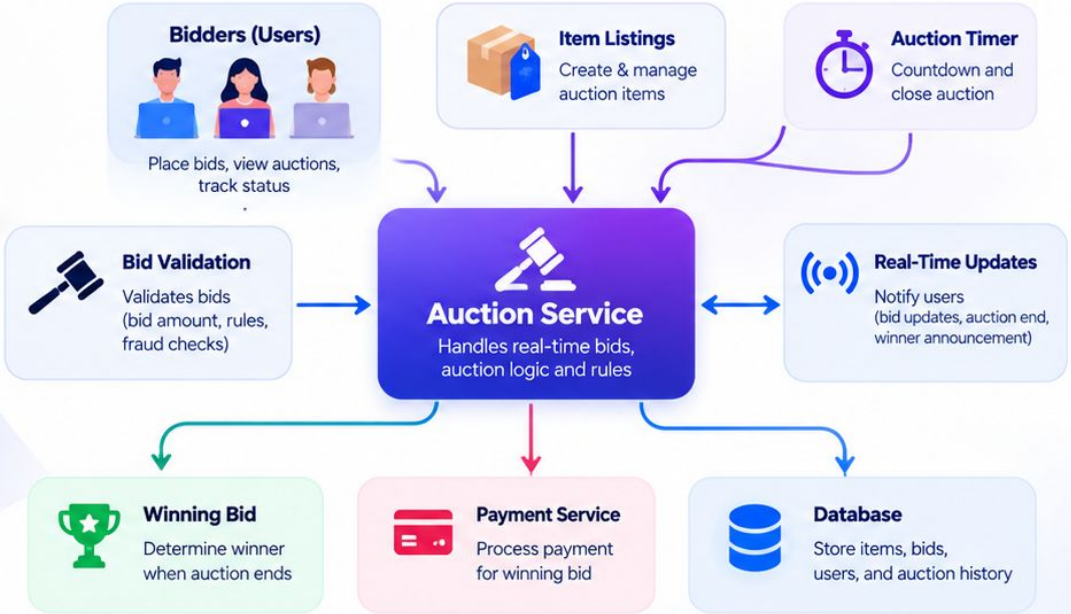
SYSTEM DESIGN

Design an Auction Platform (aka eBay)

Mastering System Design: From Basics to Cracking Interviews

Rahul Rajat Singh





What is an Auction Platform?

A digital system that enables users to **list items** and **bid in real time** within a defined time window.

Auctions promote **competitive pricing and urgency** — the highest bid at closing time wins.

The platform handles **item listings, bid tracking, real-time updates, auction lifecycle**, and **payment processing.**

1

List an Item

Seller lists an item with details and auction duration.





List Auction

2

Live Bidding

Users bid in real time within the defined time window.

Auction ends in
00:02:15

$320
2 mins ago

$300
5 mins ago

$280
8 mins ago

Place Bid

3

Auction Closes

Highest bid at closing time wins.



Winning Bid
$320



Congratulations!
You won this item.

4

Payment Processing

Secure payment completes the transaction.



Payment Successful
Transaction completed securely.

Key Objective: Deliver a secure, fair, and scalable real-time bidding experience — from listing to payment.

Functional Requirements

1. **User registration & authentication**

2. **Item listing with auction parameters**
(start/end time, reserve price)

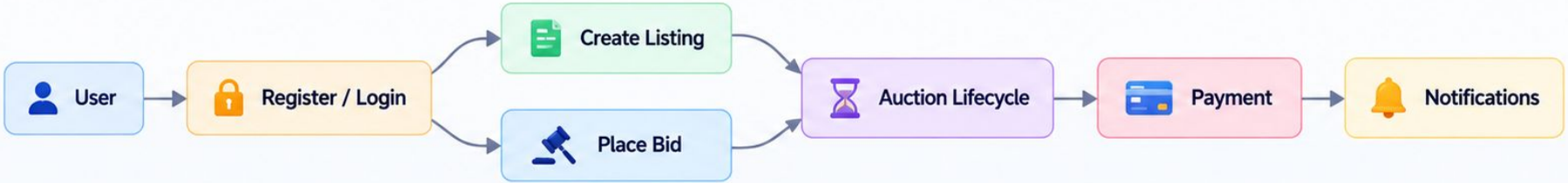
3. **Real-time bid placement and updates**

4. **Auction state transitions:**
scheduled → active → ended

Scheduled → Active → Ended

5. **Payment processing after auction ends**
(e.g., via Stripe, PayPal)

6. **Notifications:**
outbid, auction won, payment pending/complete



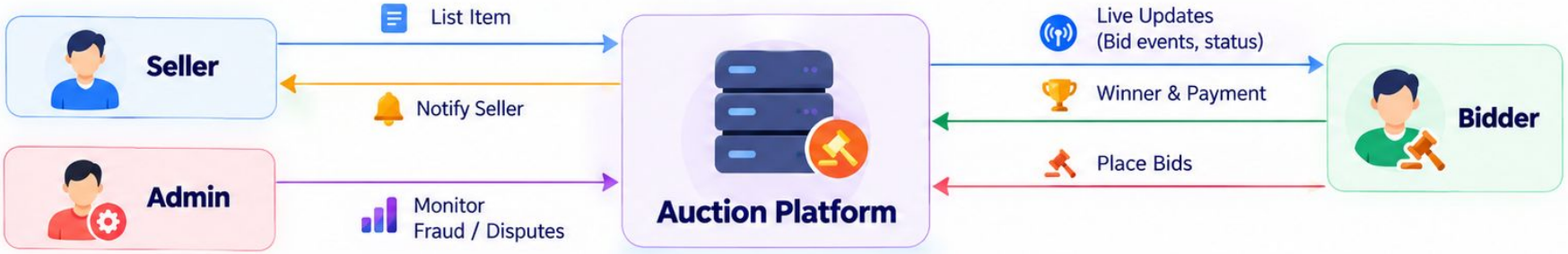
Key Actors & Use Cases

Seller
Lists items, sets pricing rules

Bidder
Places bids, gets real-time notifications

System
Handles auction logic, enforces timing, manages payments

Admin
Monitors platform activity and handles fraud or disputes



Use Cases: Auction Scenario

1. Seller lists a gaming console for a 3-day auction
2. **Multiple** bidders join and bid competitively
3. System tracks bids and updates the UI in real time
4. **Winning bidder** completes payment → seller is notified

Non-Functional Requirements

1 Performance



- Sub-second latency for bid placement and bid updates

2 Scalability



- Support thousands of concurrent auctions and real-time users

3 Security



- Secure user authentication, payment data protection, anti-bot protections

4 Availability



- Ensure high availability during peak auction times and payment flow

5 Observability



- Logging of auction and payment events, real-time monitoring dashboards

Constraints & Challenges

Real-Time Pressure

Handle bids submitted within milliseconds of closing — resolve ties & order events accurately.

Concurrency Conflicts

Simultaneous bid updates must be conflict-free and idempotent.



Live Updates

Efficient, scalable real-time delivery to all auction watchers via WebSockets/pub-sub.

Payments Workflow

Handle payment failures, timeouts, retries, and fraud detection gracefully.

Fairness & Trust

Prevent bots/sniping and ensure fair auction rules are enforced transparently.

Estimating Scale

Assumed Metrics These are reasonable, simplified estimates to help us design the system.



Users

5M

registered users, 500K DAU



Active Listings

1M

ongoing auctions at any moment.



Bids

10M

bids/day (Avg. 10 bids/auction)



Peak Activity

10K

concurrent users bidding/viewing one popular auction.



Payments

100K

completed auctions/day → 100K payment transactions/day.



5M
Registered Users



500K
Daily Active Users



1M
Active Auctions



10M
Bids / Day



10K
Concurrent Users on Popular Auction



100K
Payments / Day

Traffic Patterns & Real-Time Pressure Points

READ-HEAVY OPERATIONS



- Viewing auction listings & item details
- Real-time bid updates to watchers
- Browsing user profiles, search, filters
- **Scale:** Millions of reads per minute at peak

WRITE-SENSITIVE OPERATIONS



- Bid placement: low latency + strong consistency
- Creating new auction listings
- Finalizing auctions and triggering payments
- Lower write volume, but **time-critical near auction end**

REAL-TIME PRESSURE ZONES



- Last-second bidding: thousands of bids in final seconds
- Real-time fan-out: **pushing updates** to thousands of **watchers**
- **Closing auctions precisely** (scheduler must be accurate)
- Payment triggers: time-sensitive + third-party API dependent

TAKEAWAY | Scale for reads. Keep writes fast, consistent, and correct under pressure.

Identifying System Bottlenecks and Challenges



Auction Platform
Under Growth

1

Capacity Planning

Estimate per-service load to prevent surprise bottlenecks.

2

Real-Time Isolation

Isolate real-time components for better scalability.

3

Async Processing

Use asynchronous processing where consistency can be relaxed (e.g., email, payment receipt generation).

4

Partition Data

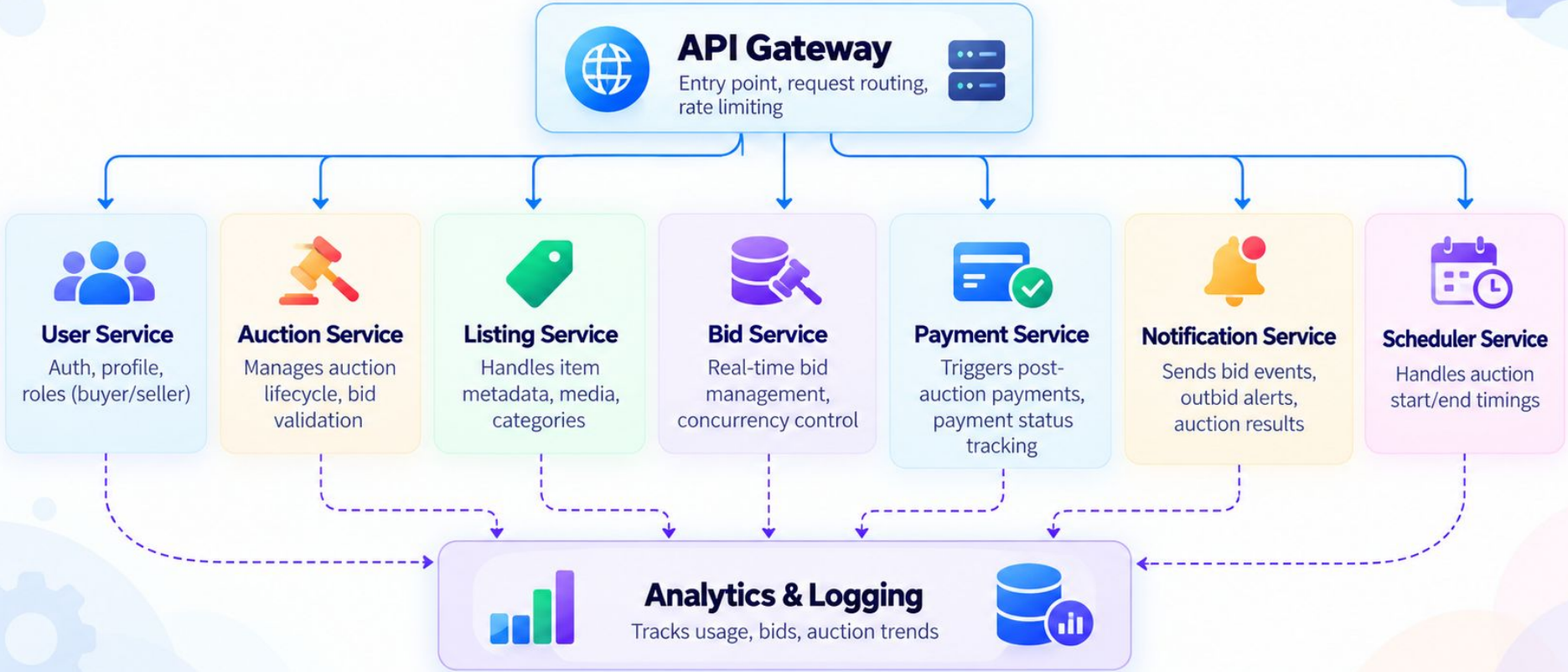
Use horizontal scaling + partitioning for high-volume data (e.g., bids, listings).

5

Hot Auction Handling

Plan for “hot” auctions — design for skewed load.

High-Level Architecture Overview and Key components



API Design — Key Endpoints

User APIs

User registration, authentication and profile management.

POST	/signup	Register a new user
POST	/login	Login user
GET	/user/profile	Get user profile

Auction APIs

Create auctions, place bids and view auction details.

POST	/auctions	Create new auction
GET	/auctions/{id}	View auction
POST	/auctions/{id}/bids	Place bid
GET	/auctions/{id}/bids	Bid history
GET	/auctions/active	List active auctions

Payment APIs

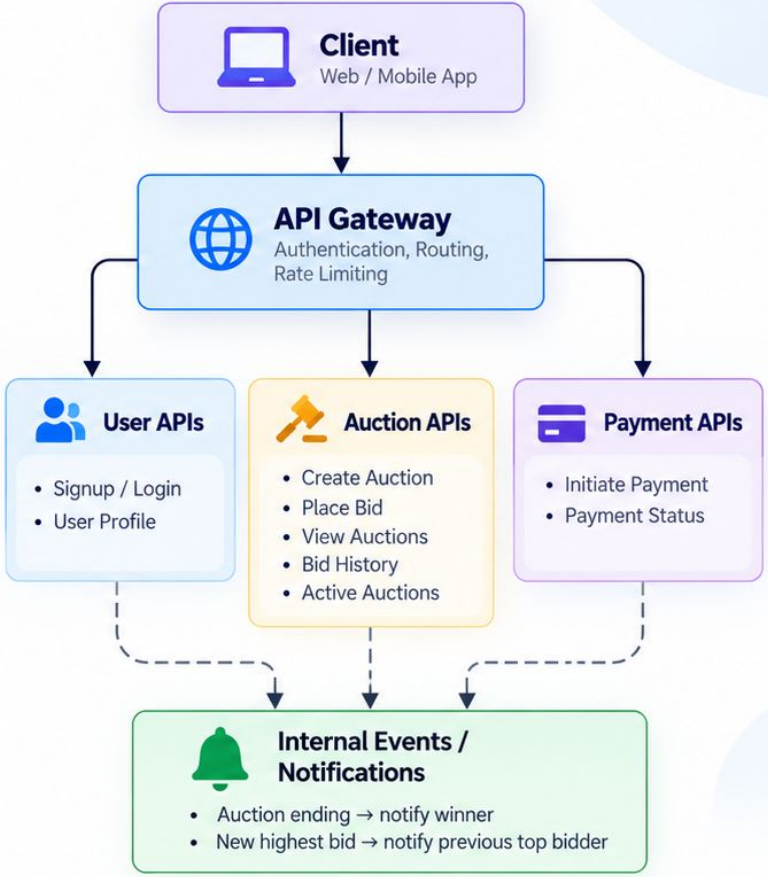
Handle post-auction payments and status tracking.

POST	/payments/initiate	Trigger post-auction payment
GET	/payments/{id}/status	Get payment status

Notification Triggers (Internal)

Trigger internal events for notifications.

Auction ending	Notify winner
New highest bid	Notify previous top bidder



Service-to-Service Communication

Communication Patterns

Synchronous (REST / gRPC)

Request / Response

- User authentication and listing fetch
- Auction → Bid → Payment trigger

Asynchronous (Pub/Sub or Event Bus)

Event Driven

- New bid placed → broadcast to watchers
- Auction ended → notify winner and trigger payment
- Failed payment → retry / alert

Sample Event Topics

`auction.ended` `bid.placed` `payment.failed` `user.registered`

Key Benefits

Loosely coupled systems | Retries | Better failure handling



Real-Time Bid Delivery (WebSockets)

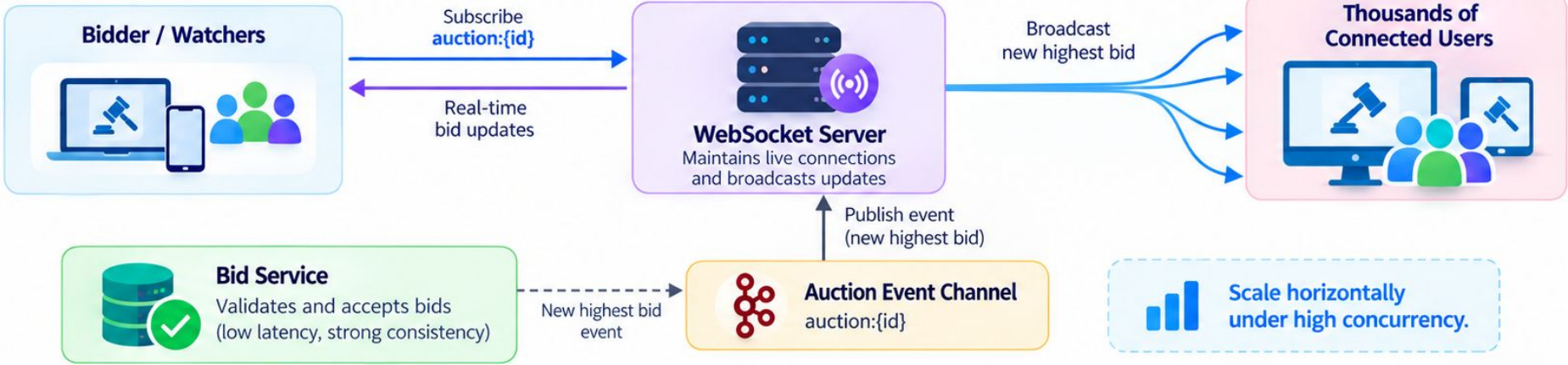
WHY WEBSOCKETS?

Low-latency, full-duplex communication

Push bid updates to all watchers instantly

HOW IT WORKS

1. Client subscribes to **auction:{id}**
2. Bid service validates and accepts bid
3. Publish the new highest-bid event
4. WebSocket server broadcasts to connected clients



Data Model (Simplified)

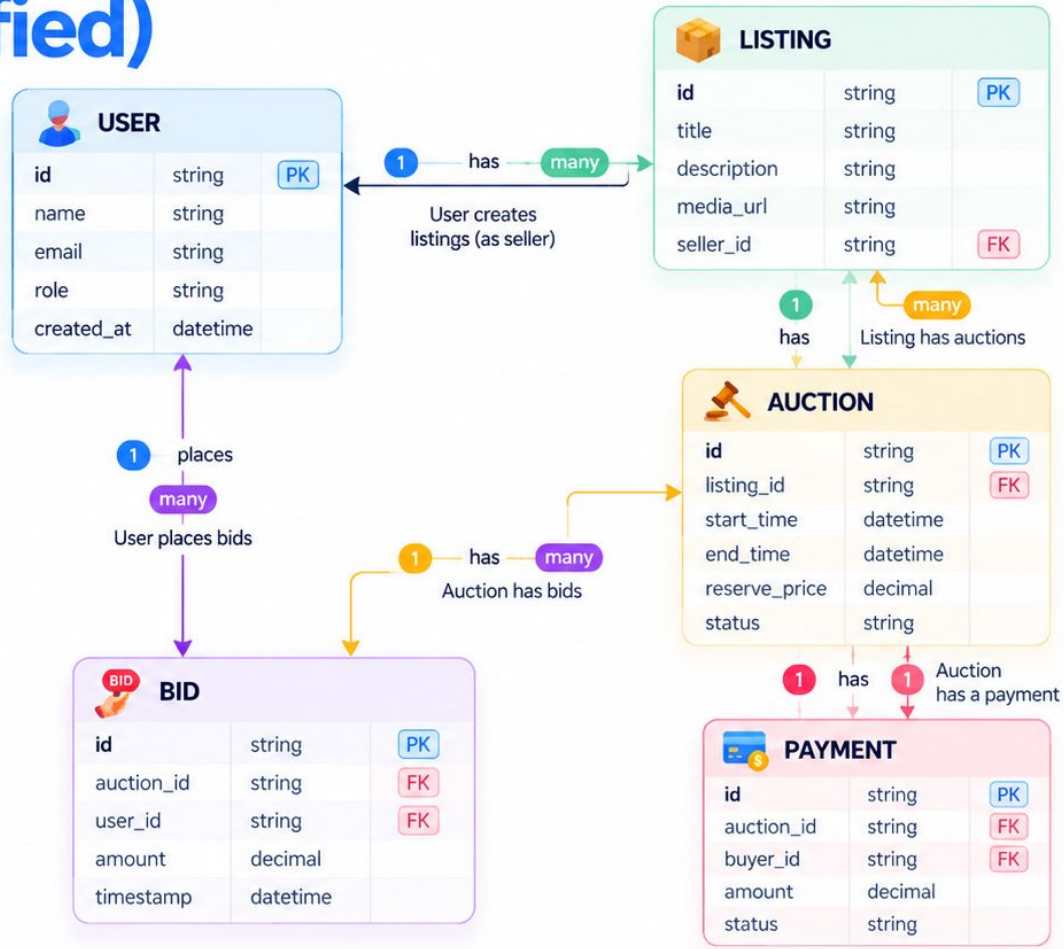
1. **USER**
   Stores information about the users (buyers/sellers).

2. **LISTING**
   Contains details about the items being auctioned.

3. **AUCTION**
   Represents an auction for a specific listing, including its start and end time.

4. **BID**
   Stores bids placed on an auction by users.

5. **PAYMENT**
   Tracks payments after auction completion.



Handling Auction Timers & Closures - Scheduled Job System for Auction Lifecycle



Responsibilities

1. Start auction at start_time
2. End auction at end_time
3. Determine winner, notify parties, trigger payment

Design Options

1. Dedicated Scheduler Service (cron/queue-based)
2. Use Delayed Jobs via message queue (e.g., SQS delay, Kafka, BullMQ)
3. Store timers in Redis + polling/expiration-based mechanism

Reliability Measures

1. Retry failed closures
2. Idempotent end-of-auction logic
3. Logging + alerts on missed timers

Instructor: Rahul Rajat Singh

Strategic Tech & Infra Decisions

Technology Stack for Scalability

Frontend
React or Vue for responsive, real-time UI.

Backend
Node.js or Java with Spring Boot for handling API requests efficiently.

Database
PostgreSQL for relational data, Redis for caching high-demand data like auction info.

Ensuring High Availability & Performance

Load Balancing
Use cloud-based load balancers to distribute traffic evenly across servers.

Real-Time Updates
WebSockets for real-time bidding and auction status updates.

Caching
Redis to cache frequently accessed data, improving performance and reducing database load.

Security Considerations

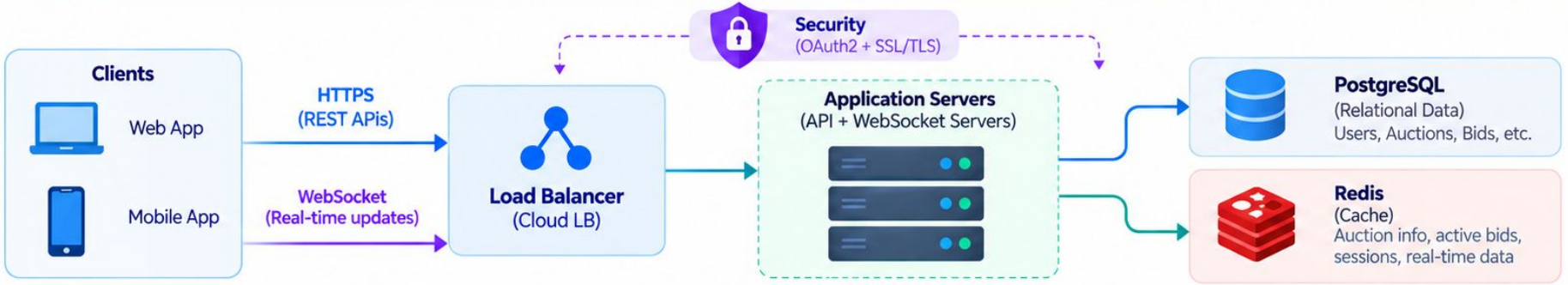
Authentication
OAuth2 for secure user login (buyers/sellers).

Data Encryption
SSL/TLS for encrypting data in transit, ensuring secure transactions.

Cost Optimization

Cloud Services
Use cloud providers' pay-as-you-go models to optimize infrastructure costs.

Autoscaling
Leverage cloud autoscaling to manage traffic surges, ensuring resource efficiency.



The Final Design - Auction Platform

